# LLM Fine-Tuning Project

Domain-Specific Q&A Model via QLoRA Fine-Tuning

## 1. Problem Statement

Fine-tune an open-source LLM (Mistral-7B or Llama-3-8B) using QLoRA to build a domain-specific Q&A assistant for finance/compliance queries. The goal is to demonstrate the ability to customize a base model for a specific task, not just prompt-engineer around it.

## 2. Objectives

- Fine-tune a 7B-class open-source LLM on a narrow, well-defined task using parameter-efficient methods (LoRA/QLoRA).
- Achieve a measurable improvement over the base model on held-out evaluation.
- Deploy the fine-tuned model behind a FastAPI backend with a simple chat UI for demonstration.
- Document the full pipeline (data, training, eval) for resume and interview use.

## 3. Tech Stack

| Layer | Tool | Purpose | Notes |
|---|---|---|---|
| Base model | Mistral-7B-Instruct or Llama-3-8B | Starting checkpoint | Instruct-tuned variant preferred |
| Fine-tuning | PEFT (LoRA) + bitsandbytes (4-bit) | Parameter-efficient training | QLoRA = LoRA + 4-bit quantization |
| Training loop | Hugging Face TRL (SFTTrainer) | Supervised fine-tuning | Handles formatting + loss |
| Compute | Google Colab / Kaggle | Free T4/P100 GPU training | Kaggle free GPU sessions |
| Serving | FastAPI + PEFT adapter loading | Inference API | Loads base + LoRA weights |
| Frontend | Next.js + Tailwind | Chat demo UI | Reuse existing stack |
| Evaluation | Custom eval script + LLM-as-judge | Accuracy / quality scoring | Compare base vs fine-tuned |

## 4. Dataset

**Sources (pick one, or blend):**

- FinQA (financial reasoning Q&A) — huggingface.co/datasets/dreamerdeo/finqa
- FiQA (financial opinion/QA) — huggingface.co/datasets/pauri32/fiqa-2018
- LegalBench (legal reasoning tasks) — huggingface.co/datasets/nguha/legalbench
- Custom: scrape public compliance/FAQ docs and convert to instruction-response pairs

**Format:** JSONL, one object per line: `{"instruction": "...", "input": "...", "output": "..."}`

**Size:** 500–2,000 cleaned, deduplicated examples. Hold out 10% (stratified) for evaluation.

## 5. Implementation Plan

### Phase 1 — Scope & Setup (Day 1)

- Finalize task definition and success metric (e.g., answer accuracy, faithfulness score).
- Set up Kaggle/Colab notebook with GPU runtime.
- Install: transformers, peft, bitsandbytes, trl, accelerate, datasets.

### Phase 2 — Data Preparation (Day 2–3)

- Download and clean source dataset(s).
- Deduplicate, filter low-quality examples, normalize formatting.

- Convert to instruction-tuning JSONL format.
- Split: 90% train / 10% eval (stratified by category if applicable).

### Phase 3 — Training Configuration (Day 3–4)

- Load base model in 4-bit using bitsandbytes NF4 quantization.
- Apply LoRA: rank 8–16, alpha 16–32, target modules = q_proj, v_proj, k_proj, o_proj.
- Training args: 3–5 epochs, batch size 4 with gradient accumulation 4–8, learning rate 2e-4, cosine scheduler.
- Use SFTTrainer from TRL with the prepared dataset.

### Phase 4 — Fine-Tuning Run (Day 4–5)

- Launch training and monitor loss curves (train + eval loss).
- Save LoRA adapter checkpoint (typically 50–300MB, not the full model).
- If loss diverges or overfits: reduce rank, add dropout, or reduce epochs.

### Phase 5 — Evaluation (Day 5–6)

- Run base model and fine-tuned model on the same held-out set.
- Score: task accuracy / exact match where applicable, plus qualitative relevance.
- Optional: use a stronger LLM (e.g., GPT-4o-mini or Claude) as a judge to score response quality 1–5.
- If eval fails to show improvement: retrain with adjusted data quality, LoRA rank, or epochs; loop back to Phase 3.

### Phase 6 — Deployment & Demo (Day 6–7)

- FastAPI endpoint: loads base model + LoRA adapter via PEFT, exposes /generate.
- Next.js chat interface: side-by-side base vs fine-tuned comparison.
- Push code + adapter weights + README (with eval numbers) to GitHub.

## 6. Evaluation Metrics to Report

| Metric | Base Model | Fine-Tuned Model |
|---|---|---|
| Task accuracy / exact match | — | — |
| LLM-judge quality score (1–5) | — | — |
| Response relevance | — | — |

Fill in after evaluation — this table becomes the results section.

## 7. Risks & Mitigations

- **Free GPU session timeout mid-training:** checkpoint every N steps and resume from checkpoint.
- **Small dataset overfitting:** use dropout in LoRA config, limit epochs, and monitor eval loss.
- **Base model already performs well (no visible improvement):** narrow the task further or pick a harder/rarer domain.

## 8. Resume Line

*"Fine-tuned Mistral-7B using QLoRA on a custom [finance/legal] Q&A dataset (X examples), achieving [X%] improvement in [metric] over the base model on held-out evaluation; deployed via FastAPI with a Next.js comparison interface."*

## 9. Timeline Summary

| Day | Phase | Deliverable |
|---|---|---|
| 1 | Scope & Setup | Environment ready, task defined |
| 2–3 | Data Preparation | Cleaned JSONL dataset, train/eval split |
| 3–4 | Training Config | LoRA config finalized |
| 4–5 | Fine-Tuning Run | Trained adapter checkpoint |
| 5–6 | Evaluation | Base vs fine-tuned comparison results |
| 6–7 | Deployment | FastAPI + Next.js demo, GitHub repo |